— FULL-STACK DEVELOPER · 0–2 YEARS · NODE.JS, REACT & AI AGENTS

Gold Loan Portal, With an AI Agent.

Take-home technical assignment · 48 hours

Build a production-minded gold-loan intake portal: a validated full-stack app with exact financial maths, plus an AI agent that helps borrowers apply by calling your backend tools instead of guessing. We want to see how you build, and how carefully you check what AI writes for you. Expect about 10–14 hours of focused work. A small, correct solution beats a big, unfinished one.

48 hrs	~12 hrs	3	100
Time limit	Expected real effort	Agent tools to build	Point rubric

FROM THE TEAM THAT BUILDS FOR

Foot Locker · Puma · Dell · Ackermans · DealShare

Submit: GitHub repo + README + AI_LOG

Start Building →

THE BRIEF

One Portal. Two Ways To Apply.

A lending partner needs a web portal where field agents and borrowers can get a preliminary gold-loan offer. Borrowers can fill a **guided form** or **chat with an AI assistant**. Both paths must run on the same validated backend and produce identical numbers.

A
Core platform REQUIRED
4 endpoints, maths, validation, 3-step form, admin table

B
AI agent REQUIRED
A 3-tool Loan Assist chat with 5 test conversations

C
Bonus PICK ANY
Hardening, slip extraction, Docker, CI

D
Ship
README, AI_LOG, tests

What We're Really Testing

Correctness Under Pressure
Money maths that is exact, rounded consistently and covered by tests. Not "roughly right".

API Discipline
Schema validation on every request, the right status codes (400 / 404 / 409) and clear, consistent error messages.

Agent Engineering
An LLM that calls typed tools, never does arithmetic itself, asks before acting, and can't be talked out of its rules.

Data Integrity
The server re-calculates every number itself and rejects repeat applications from the same mobile within 7 days.

Product Sense
A clean, responsive, accessible UI with live feedback. Something a field agent would actually enjoy using.

Critical Use of AI Tools
Use Cursor, Claude Code or Copilot freely, then show us where the AI was wrong and how you caught it.

PART A · CORE PLATFORM REQUIRED

Backend API & Business Rules.

LAYER	EXPECTED
Backend	Node.js + Express (or Fastify / NestJS), TypeScript preferred, Zod or Joi schemas
Database	PostgreSQL preferred (Prisma / Drizzle / Knex), MongoDB accepted. Include migrations and a seed script
Frontend	React (Vite or Next.js), with any component library or Tailwind

ENDPOINT	BEHAVIOUR
`GET /api/v1/loan-schemes`	Active plans with id, name, annual interest rate, tenure and max LTV (never above 75%). Seeded from the DB.
`POST /api/v1/quotes`	A pure calculation with no DB write. Returns pure gold, gold value and the eligible loan per plan. The UI calculator and the AI agent both use it.
`POST /api/v1/leads`	Validates, re-computes the quote server-side (never trust client numbers), de-duplicates, stores the lead with status `SUBMITTED` and returns `201` with an `applicationId`.
`GET /api/v1/leads`	All leads, newest first. Mobile numbers are always masked in responses. Pagination is a bonus.

Validation → 400

- Name 2–60 characters, letters and spaces only
- Mobile: Indian 10-digit, `^[6-9]\d{9}$`
- 0 < net ≤ gross ≤ 1000 g
- Karat must be one of 18 / 22 / 24. Plan id must exist (else 404)

Duplicates → 409

- The same mobile within 7 days returns `409 Conflict`, with the existing `applicationId`
- A simple lookup before inserting is fine. Making it safe for simultaneous requests is a bonus

Bonus: Harden It

- `Idempotency-Key` header on POST /leads, so retries are safe
- A status workflow (SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED) with an audit log

Error Contract

- Every error body has the same shape, for example `{ error: { code, message, fields[] } }`
- Return 500s as JSON without stack traces. Handle all async errors

PART A · MATHS, UI & TEST CASES

Exact Money. Every Time.

```
pureGoldGrams = netWeightGrams × (karat / 24)          // 22K → 22/24 = 91.67%
goldValue     = pureGoldGrams × rate24kPerGram
eligibleLoan  = floor( goldValue × min(plan.maxLtv, 0.75) )
// Loan is in whole rupees and never above 75% LTV
// Store money as integer paise or DECIMAL, never as float rupees
```

Seed these two plans. Use a mock 24K rate of **₹7,000/g** so your results can be checked against the table below.

PLAN ID	NAME	INTEREST (P.A.)	MAX LTV	TENURE
`PLAN_BULLET_01`	Bullet Repayment	12.0%	70%	12 months, interest + principal at maturity
`PLAN_EMI_01`	Monthly EMI	10.5%	75%	12 months, equal monthly instalments

#	INPUT	EXPECTED
1	45 g net · 22K · EMI	pure 41.25 g · value ₹2,88,750 · loan **₹2,16,562** · `201`
2	45 g net · 22K · Bullet	value ₹2,88,750 · loan **₹2,02,125** (70% LTV)
3	10 g net · 18K · EMI	pure 7.5 g · value ₹52,500 · loan **₹39,375**
4	net 52 g > gross 50 g	`400` with field error on `netWeightGrams`
5	mobile `5876543210` or `98765`	`400` with field error on `mobileNumber`
6	Same mobile again within 7 days	`409` with the existing `applicationId`
7	BONUS Same request, same `Idempotency-Key`	The original `201` response, replayed
8	BONUS 2 parallel requests, same new mobile	Exactly one `201` and one `409`

PART A · THE WEB PORTAL REQUIRED

A Portal Field Agents Enjoy Using.

One responsive single-page app (React with Vite or Next.js) with a three-step application flow and an admin view. Keep it clean and fast, with live feedback on every field.

01
Details
Name, mobile, gross and net weight, karat. Inline validation as you type

02
Live quote
Pure gold, value and loan per plan, as selectable plan cards (call /quotes, debounced)

03
Confirm
Review, submit, then a success screen with the Application ID. Friendly 409 message

04
Admin view
Leads table: masked mobile (`9876XXXX10`), net weight, plan, loan, status, filters

Live, Not Laggy

Re-quote as the user types, debounced at about 300 ms. Show pure gold, gold value and the loan for each plan, and highlight the selected plan card.

Errors People Understand

Show inline field errors from the API's `fields[]` . A 409 becomes "You already applied on 3 Oct, application GL-1042", not "Error 409".

Admin Table

Newest first, with the mobile masked as `9876XXXX10` . Show net weight, plan, loan amount and status, plus a simple filter by plan.

Works on a Phone

Field agents use mobiles. The flow must work at 375 px wide, with sensible input types (numeric keypad for weights and mobile).

GROUND RULES

Any AI coding tool is allowed and encouraged. Never commit API keys: use `.env` and ship a `.env.example` . If you have no paid LLM key, a free tier or a local model through Ollama is fine. Document which one you used. The agent's design matters more than which model it runs on.

PART B · AI LOAN ASSIST AGENT REQUIRED

An Agent That Uses Tools, Not Guesswork.

Add a chat panel where a borrower can say *"I have 45 grams of 22 carat jewellery, how much can I get?"*. The agent collects the missing details conversationally, quotes through your backend, and files the application, but only after the user explicitly confirms.

TOOL (FUNCTION CALLING)	DOES	RULE
`get_loan_schemes()`	Lists plans and rates	The agent may only describe plans returned by this tool
`calculate_quote(net, gross, karat)`	Calls `POST /quotes`	The LLM must **never** do the maths itself. Every number comes from this tool
`submit_application(payload)`	Calls `POST /leads`	Runs only after a confirmation step in the UI that shows the full summary

Structured & Validated

Tool arguments are validated with the same Zod schemas as the API. When a tool call is invalid, the agent gets the error back and asks the user to fix it.

Guardrails

Stays on gold loans. Gives no investment advice and makes no promises of approval. Politely declines anything off-topic.

Privacy

Mobile numbers are masked in logs. No API keys in the browser, so all LLM calls go through your backend.

Same Truth, Two Paths

A lead filed by the agent must be identical to one filed through the form. Both pass through the same service layer.

TEST CONVERSATIONS

Write **5 test conversations** with their expected outcomes. A script (`npm run eval`) is great; a written checklist with screenshots is fine. Cover:

- a happy path, through to submission
- a missing field the agent asks for
- net weight greater than gross
- a duplicate within 7 days
- an off-topic request

Bonus: prompt injection, Hinglish, the user changing their mind, 10+ automated evals.

BONUS · PICK ANY OPTIONAL

Finished Early? Go Further.

Only start these once Parts A and B work end to end. Pick what interests you. One bonus done well is worth more than four started.

B1 · Harden the Backend BONUS

Duplicate checks that hold under simultaneous requests, an Idempotency-Key, a status workflow with an audit log, and a cached gold-rate endpoint.

B2 · Smarter Agent BONUS

A 4th tool, `check_existing_application`. Add prompt-injection tests, streamed replies and tool-status chips ("Calculating quote…").

B3 · AI Extras BONUS

Read gross weight, net weight and karat from a photo of an appraisal slip, or let admins filter leads in plain English through a safe tool.

B4 · Ops Ready BONUS

`docker compose up`, a GitHub Actions run for tests, a rate limit on /leads, an ARCHITECTURE.md and a 2-minute demo video.

HOW BONUS POINTS WORK

Each bonus can add up to **+5**, with a maximum of **+15** in total. Bonus points never make up for missing required work. Mention every bonus you attempted in your README, even if it's unfinished.

DELIVERABLES & SCORING

What to Submit, How We Score It.

SUBMIT

- A GitHub repo where `npm install`, `npm run dev` and `npm test` all work
- **README.md**: setup, env vars, and what you'd do with more time
- A `.env.example`, and no committed keys

AI_LOG.MD

- Which AI tools you used, and for what
- 2 exact prompts: one for validation, one for the agent tools
- **1 place the AI got it wrong**, how you spotted it, and the test that now catches it

CRITERION	WEIGHT	WHAT SCORES WELL
Maths & validation	25	Test cases 1–6 pass, money in whole rupees or paise, LTV enforced on the server
AI agent	25	3 working tools, no maths done by the model, confirmation before submitting, stays on topic
Full-stack UX	20	Clean three-step flow, live quote, usable chat and admin table, works on mobile
Tests & test conversations	15	Unit tests for the maths, API tests for 400 and 409, all 5 conversations covered
API & errors	10	Correct status codes, consistent error body, async errors handled
AI_LOG & README	5	An honest, specific AI mistake. Setup works first time

Bonus tasks can add up to +15, but only once the required parts work. We'd rather see 6 things done well than 12 half-done. Stuck? Write down what you tried in the README. That counts too.